# Characterizing Type Feedback in Just-in-Time Compilation

Sebastián Krynski skrynski@gmail.com
Filip Říha rihafili@fit.cvut.cz
Filip Křikava filip.krikava@fit.cvut.cz
Jan Vitek vitekj@icloud.com
Oliver Tušla tuslaoli@fit.cvut.cz

## The Problem

- JIT compilers speed up dynamic languages by **recording runtime observations**, then **specializing** the compiled code to them.
- Recording **slows the interpreter** — and the cost is often **wasted**:
  - many functions never get hot enough to compile,
  - execution falls back to the interpreter after a **deoptimization**,
  - short-running scripts never amortize it.

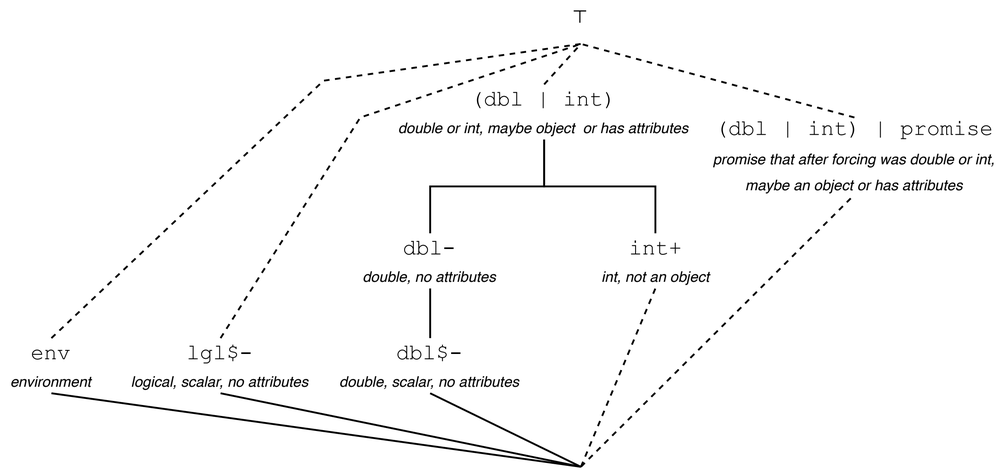
1. **How** does the compiler actually **use** recorded feedback — and how much is left unused?
2. Can we **record less** without hurting **peak performance**?

### Our approach

1. Instrument **Ř**, a JIT compiler for **R** — laziness and reflection make recording especially aggressive.
2. **Snapshot and classify every slot** during compilation.
3. Do this on a corpus of **14 benchmarks**, **3 Kaggle notebooks**, and the **Recommenderlab** library – thousands of slots.

## Type Feedback 101

- Three kinds recorded: **types**, **call targets**, **branch decisions** — we study **types**.
- A type descriptor is a nominal type plus suffixes: ($) = **scalar** (length-1 vector), (-) = **no attributes**, (+) = **attributes but not an object**, bare = **maybe an object**.
- These form a **lattice**. Each instruction that may be speculated has a **slot** recording the type it observes; a function's slots make up its **feedback vector**. Repeated observations on a slot **merge** (⊔) and lose precision (**pollution**).




The compiler reasons with **three** types per instruction:

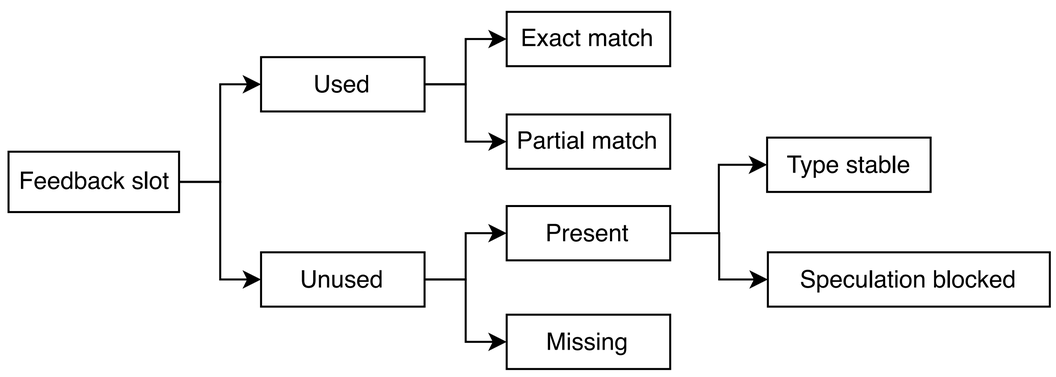
- **Inferred** (IT) — known statically by dataflow analysis,
- **Observed** (OT) — recorded into the slot at runtime,
- **Expected** (ET) IT ⊓ OT — what it actually speculates on.

Speculation pays off **only** when ET is **more precise than** IT.

## What the Compiler Actually Uses

We snapshot every feedback slot during compilation and classify each as:

- **Used** — speculated on (exact match, or narrowed / widened).
- **Unused but present** — survived, but the type was **stable** (inferred already matches observed) or speculation was **blocked**.
- **Unused and missing** — **eliminated** during optimization.

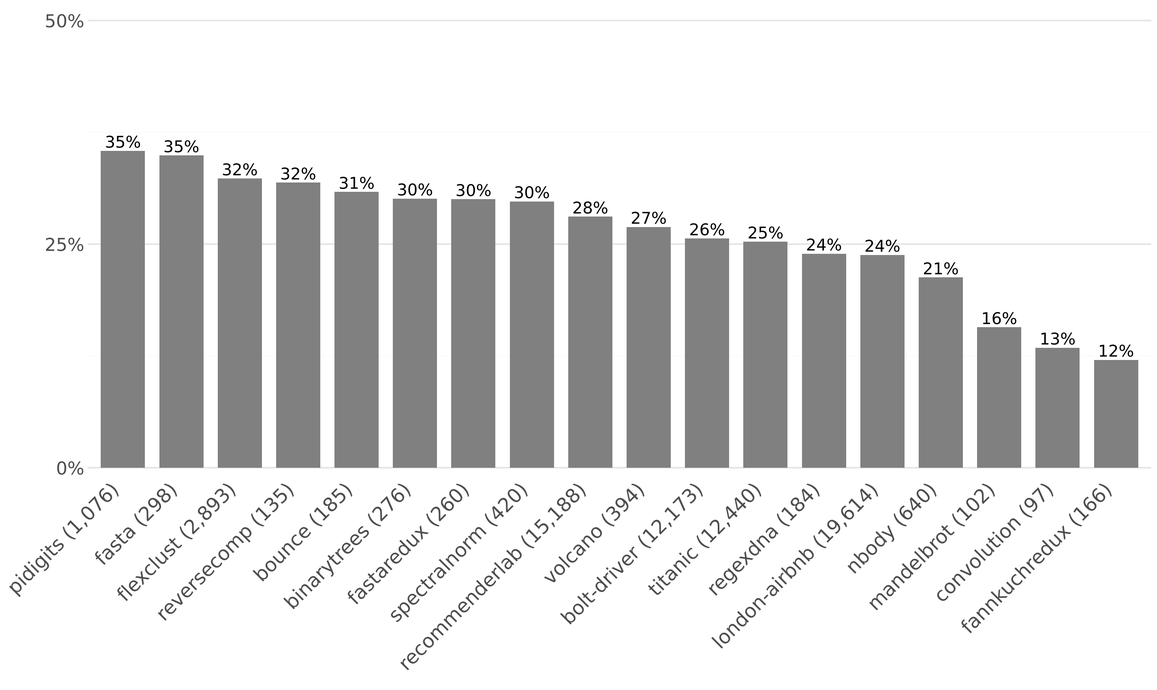



**up to 59%** of non-empty feedback slots are **never used** by the compiler

**up to 1.6×** interpreter recording overhead (mean 1.2×)

This waste is mostly explained. Of these unused slots:

- **≈43%** are **dead code** – eliminated before they could be used;
- most of the rest are **type-stable** – the inferred type already matches the observed one, so speculation cannot help.

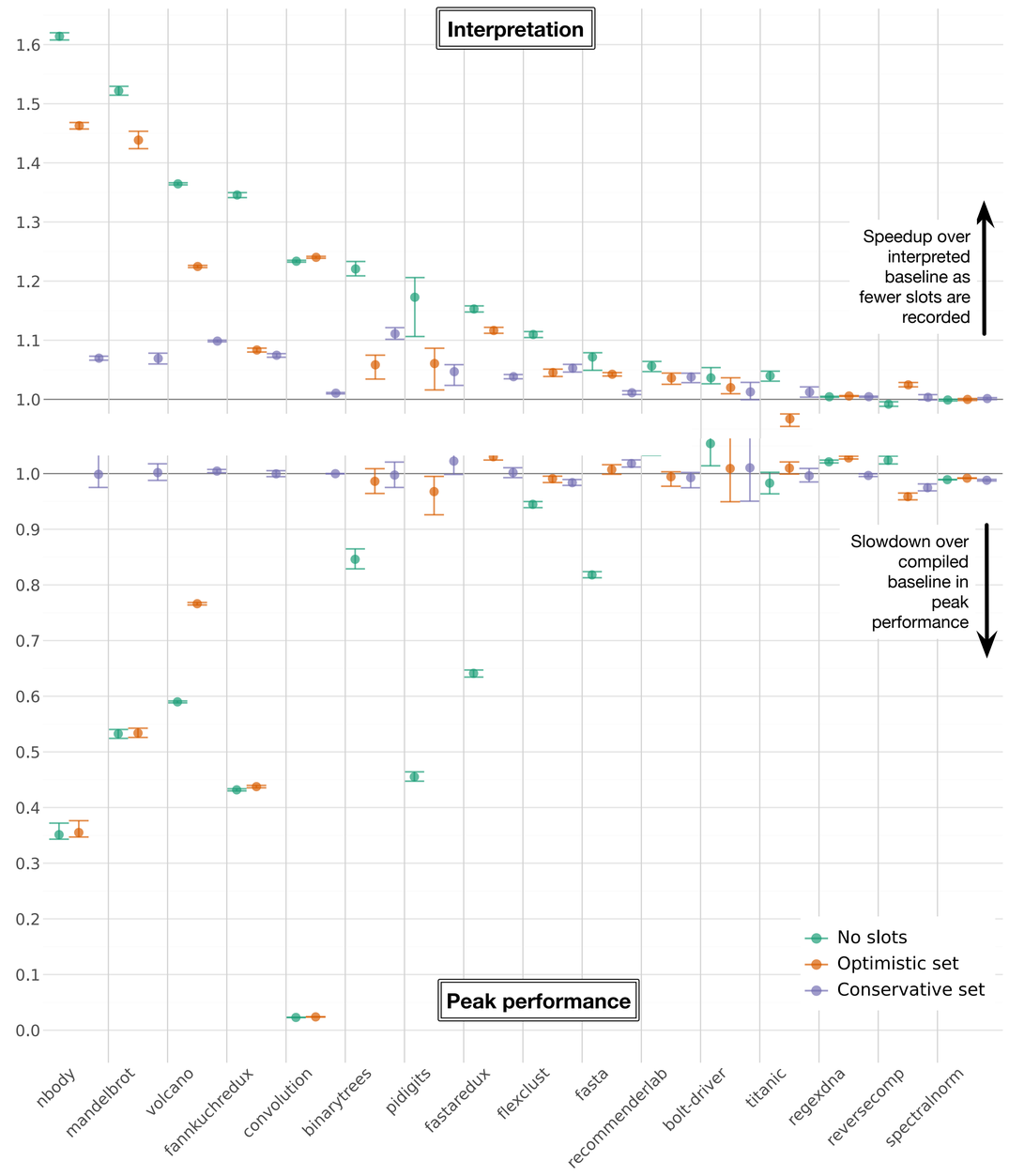



*Ratio of **used** slots out of non-empty slots, per program – only about a quarter are ever used.*

## How Much Could We Save?

Oracle configurations are upper bounds on what selective recording could achieve:

- **Baseline** — record **every** slot (what Ř does today).
- **Conservative set**
  - keep every slot the compiler **uses at any point** during compilation.
  - the interpreter **speeds up** – there is less to record,
  - **peak performance stays at baseline** – the same slots were used for optimization.
- **Optimistic set**
  - keep only slots whose speculation **survives into the final optimized code**.
  - the interpreter speeds up **even more**,
  - **peak performance drops** on some compute-heavy programs
    → some slots that were used for intermediate optimizations were removed.




*Interpretation speedup (top) and peak performance (bottom), normalized to the all-slots baseline.*

### Takeaway

- JIT compilers record **far more feedback than they use**.
- **Dead-code** and **type-stable** slots are wasteful.
- Recording **far less is possible without sacrificing optimization quality**.
  → Future work: **predictive, selective recording**.

Attend the talk!
Thursday at 14:22

Read the paper!
10.4230/LIPIcs.ECOOP.2026.16

FUNDED BY GAČR CZECH SCIENCE FOUNDATION

This work is supported by the Czech Ministry of Education, Youth and Sports under program ERC-CZ, grant agreement LL2325, as well as by the Czech Science Foundation Grant No. 23-07580X.

FACULTY OF INFORMATION TECHNOLOGY CTU IN PRAGUE
CHARLES UNIVERSITY
PRL@PRG
D3S